PRODUCT BACKLOG

CBL GAME DEVELOPMENT: MINESWEEPER

By Gulce Ozhan (2457288) and Oney Khambhadia (2422301)

FINAL VERSION OF THE BACKLOG

#	Name	How to Demo	Notes
1	Display game board	Launch the app; a grid of clearly distinguishable covered cells and a game control panel above are displayed in the GUI.	Display a grid of clickable cells, game control buttons, time count and mine count labels. Cells grid is modelled as a 1D array of size row x col. Row and col indices calculated by arithmetic (/ rowSize and mod colSize).
2	Start a new game	Launch the app; a fresh covered board is created and the game is ready. New game and level selection buttons on the control panel restart the game.	Reset the board and randomly place mines. At the same time, keep a count of neighbouring mines for each cell. Mine count is read from the properties file per difficulty level.
3	Reveal a (numbered) cell	Click a covered cell (which was numbered); the cell is revealed and shows the number of mines in the surrounding cells.	A cell has a ContentType & IconType, which are used to determine the appropriate numbered cell icon to display. A cell may have 0 to 8 mines in its immediate neighbourhood.
4	Reveal empty areas	Click a cell with no adjacent mines; nearby empty cells are automatically revealed.	Recursively reveal connected empty cells and their boundaries of numbered cells.
5	Flag a suspected mine	Right-click a covered cell; the cell is marked with a flag. Right-click again to remove.	A flagged cell means the player suspects it is a mine cell. Each flagging decreases mine count by 1. Right-clicking to revert flagging.
6	Detect a mine	Click a cell containing a mine; the game reports a loss and reveals the mines.	Stop normal gameplay after a mine cell is clicked. Smiles changes to ‘sad’, timer and mine count freeze and status displayed at the bottom.
7	Detect a win	Reveal all cells that do not contain mines and flag all the cells that have mines; the application reports that the player has won.	Check whether all non-mine cells are revealed and mined cells are correctly flagged. Smiley changes to ‘cool’, timer and mine count freeze, status displayed at the bottom.
8	End the game	Win or lose a game; the result is displayed. A new game can be started or game terminated.	Handle both win and lose cases as above. Restart a new game by clicking panel buttons, or terminate the game by clicking top-right X.

9	Track game time	Start a game; a timer begins and stops when the player wins or loses.	Display elapsed playing time. Use javax.swing.Timer with snippet:
			timer = new Timer(1000, e -> {
			seconds++;
			pnlTopControl.setTime(seconds);
			});
			timer.start();

10	Display remaining mines	Place or remove flags; the displayed mine counter updates accordingly.	Display total mines − placed flags. Placing a flag decreases mine count by 1.
11	Configure game settings and externalize	Change settings in a .properties file and launch the game; the configured values are used. In JAR usage config is externalizable.	Configure board dimensions, number of mines, difficulty, pixel size, path to icons folder. When executable JAR is used, placing a new config file in the same folder overrides JAR's config.

ADVANCED TOPICS OF CHOICE

GIT AND GITHUB

- Git is a version control system. It keeps track of changes you make to your code. You can create commits (saved checkpoints), create branches to work on features separately, and go back to previous versions if something goes wrong. Git works locally on your computer.
- GitHub is a website/service that stores Git repositories online. It allows you to back up your code and collaborate with other people. Developers can push their Git changes to GitHub and pull changes made by teammates.

PROGRAM CONFIGURATION: JAVA .PROPERTIES FILE

A Java .properties file is used to store program configuration settings outside the source code. The settings are stored as key-value pairs, such as `mine.count.beginner=10` or `cell.size.pixels=32`. The Java program can read these values using the `Properties` class and use them at run-time. This allows for changing settings without modifying or recompiling the Java code, making the program easier to configure and maintain.

SUMMARY OF MINESWEEPER GAME

For the CBL project we have developed the Minesweeper game. Minesweeper is a game played on a grid of hidden cells, some of which contain mines. The goal is to reveal all cells that do not contain mines without clicking on a mine. When a safe cell is revealed, it displays a number indicating how many mines are in the surrounding cells; if there are no nearby mines, neighbouring empty cells are usually revealed automatically. Players can flag

cells that they believe contain mines to help keep track of dangerous locations. The player wins when all non-mine cells have been revealed and loses if a mine is revealed. The main challenge is using the numbered clues and logical reasoning to determine where the mines are located.

For Minesweeper, the main mechanics can be as follows:

- **Revealing cells**: the player clicks a hidden cell to reveal it.
- **Mines**: revealing a mine causes the player to lose.
- **Number clues**: a revealed safe cell shows how many mines are in its surrounding cells (between 1–8).
- **Empty-cell expansion**: revealing a cell with no adjacent mines automatically reveals nearby safe cells.
- **Flagging**: players can flag cells they suspect contain mines.
- **Winning:** the player wins when all non-mine cells are revealed.

Some screenshots of different game states from the implemented game are shown below:

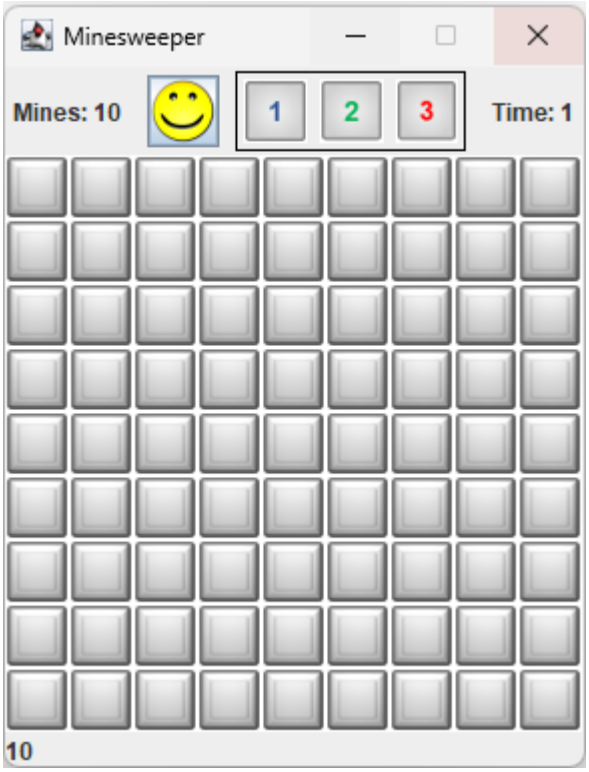


Figure 1. New game



Figure 2. Continue playing



Figure 3. Game is won



Figure 4. Game is lost with a wrong flag

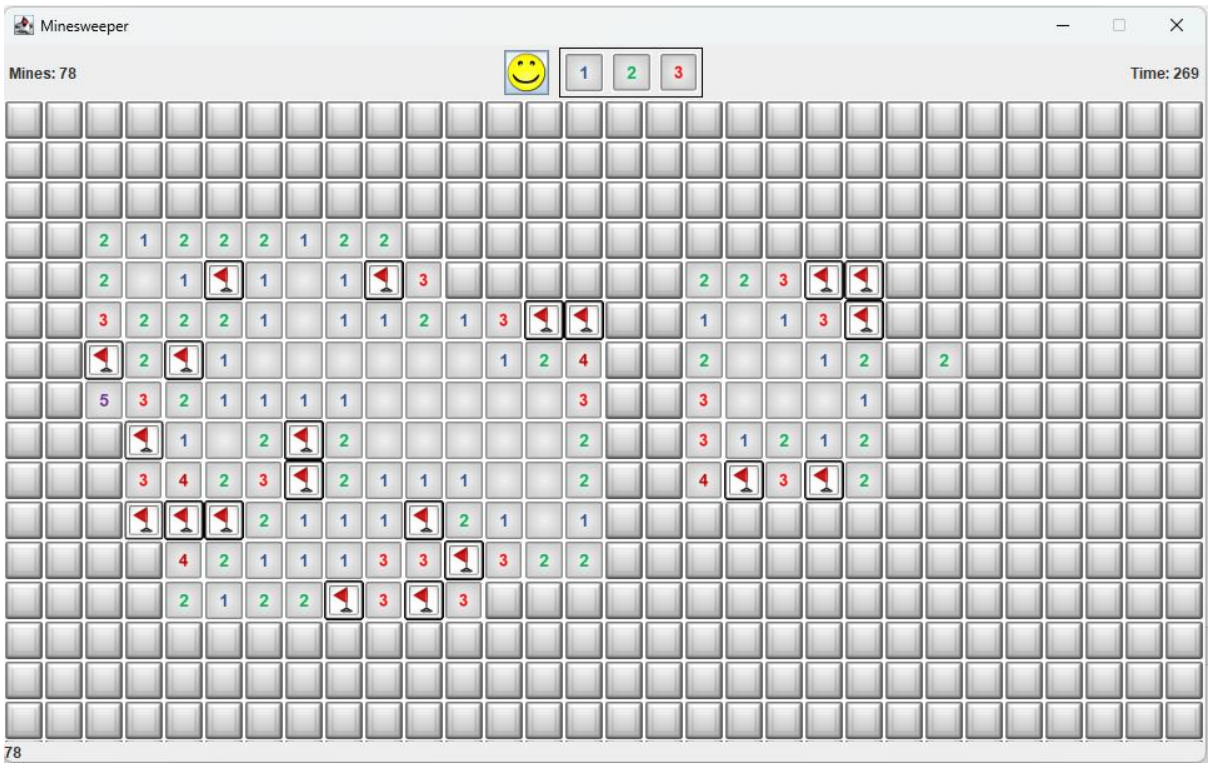


Figure 5. Continue playing (difficulty 3 on a 16x30 grid with 99 mines)